What I have learn on Monday

1. Python Basics

- **Variable:** A name that stores data.
- **Constant:** A value that should not change.
- **Data type:** The kind of data, such as a string, integer, or Boolean.
- **Comment:** Text that explains code and is ignored by Python.

2. Functions

- **Function:** A block of code that performs a specific task.
- **Parameter:** A variable listed in a function definition.
- **Argument:** A value passed to a function.
- **Return value:** The result sent back by a function.

Functions help you avoid repeating code and make programs easier to organize.

3. Modules, Packages, and Libraries

- **Module:** A Python file that contains reusable code.
- **Package:** A group of related Python modules.
- **Library:** Reusable code that helps perform specific tasks.

Using modules and packages helps organize large programs.

4. Object-Oriented Programming

Object-oriented programming, or OOP, organizes code using classes and objects.

- **Class:** A blueprint for creating objects.
- **Object:** An instance created from a class.
- **Attribute:** Data that belongs to an object or class.
- **Method:** A function that belongs to a class.

5. PEP 8 Style Rules

PEP 8 is a guide for writing clean and readable Python code.

- **Indentation:** Use spaces for each indentation level.
- **Line length:** Keep lines reasonably short, usually no more than 79 characters.
- **Imports:** Put import statements at the top of the file.
- **Naming conventions:**
    - Use snake_case for variables and functions.
    - Use UPPER_CASE for constants.

    - Use PascalCase for classes.
- **Whitespace:** Use spaces around operators and after commas.
- **Blank lines:** Use blank lines to separate functions and classes.
- **Comments:** Write clear comments when they are helpful.

6. Debugging

Debugging means finding and fixing errors in code.

Common types of errors

- **Syntax error:** The code breaks Python's grammar rules.
- **Runtime error:** An error occurs while the program is running.
- **Logic error:** The program runs but produces the wrong result.

Debugging tips

- Read the error message and check the line number.
- Use print() to check variable values.
- Check the data types of variables.
- Test one small part of the program at a time.
- Use logging to record information about the program.
- Test the code again after making a change.

7. Tips for Writing Good Code

- Use meaningful names for variables and functions.
- Keep code simple and easy to read.
- Break large programs into smaller functions.
- Avoid repeating the same code.
- Avoid hardcoding values that may need to change.
- Add comments when they are useful.
- Follow PEP 8 rules.
- Use modules to organize code.
- Handle errors when necessary.
- Test your code regularly.
- Write tests to check that the program works correctly.